

Cahier de recette

Projet : L2F – Algorithme de Déquantification par Élagage d'Arbre

Statut : Document de référence pour validation

Encadrant : Mahé Gaël

Membres : Rzaigui Adem , Mahleb Chakib Mohamed Taher , Kerrad Arezki

Rédacteur : Rzaigui Adem, Mahleb Chakib

1. INTRODUCTION

1.1. Objectifs et méthodes

Ce document définit les critères d'acceptation de l'application scientifique de déquantification.

2. CONCEPTS DE BASE

La déquantification repose sur l'idée qu'une série temporelle $x(n)$ possède une structure statistique (représentée par la matrice P) qui permet de lever l'ambiguïté introduite par la suppression d'un bit de poids faible.

3. DESCRIPTION DE LA FOURNITURE

La livraison finale doit impérativement comprendre :

1. **Code source complet** (C).
2. **La configuration** utilisée lors de l'exécution du programme.
3. **Scripts de compilation/exécution** (Makefile).
4. **Notice technique** (README) détaillant la configuration des dépendances graphiques.
5. **Rapport de test** prouvant la réussite sur le jeu de données fourni.

4. MOYENS D'ESSAI ET OUTILS

- **Matériel** : PC standard (Windows/Linux/Mac).
- **Logiciels** : Compilateur GCC (C) .
- **Bibliothèques** : SDL pour le rendu graphique de l'arbre.

5. CONFORMITÉ AUX SPÉCIFICATIONS GÉNÉRALES

ID	Libellé de l'exigence	Critère d'acceptation
GEN-01	Multiplateforme	L'application tourne sous Windows, Linux et Mac.
GEN-02	Stabilité	Pas de fuite mémoire ou de crash sur de longues séries.
GEN-03	Ergonomie	Interface graphique claire montrant l'évolution de l'arbre.

6. CONFORMITÉ AUX SPÉCIFICATIONS FONCTIONNELLES (TESTS)

6.1. GROUPE DE TEST N°1 : PORTABILITÉ ET ENVIRONNEMENT D'EXÉCUTION

- **Identification** : TEST-TECH-05 (Compatibilité Multi-OS)
- **Description** : Vérifier que le code source peut être compilé (en C)) et exécuté sur les trois architectures majeures sans modification du code.
- **Procédure** :
 1. **Sous Linux (Ubuntu/Debian)** : Exécuter le Makefile . Vérifier que l'interface graphique (SDL) s'ouvre correctement.
 2. **Sous Windows (10/11)** : Utiliser un terminal (PowerShell/CMD) pour lancer l'exécutable .exe produit.
 3. **Sous macOS (Intel/Silicon)** : Vérifier que les permissions d'exécution et les bibliothèques graphiques sont correctement liées.
- **Résultat attendu** : * Le programme se lance et traite les données redu avec les mêmes performances et le même rendu visuel sur les trois systèmes.
 - **Zéro erreur de segmentation** ou de "Library not found" au démarrage.

6.2. SCÉNARIO DE TEST N°2 : IMPORTATION ET PARSING

- **Identification** : TEST-FUNC-LOAD
- **Description** : Vérifier que le programme lit sans erreur la série quantifiée et la matrice P.
-Vérifier que le programme auxiliaire transforme correctement les données sous-quantifiées
- **Résultat attendu** : Affichage console : "Matrice P (représentée avec une table de hachage) de taille N chargée. Série de M points chargée." Avec affichage de leurs valeurs pour vérifier que la lecture est correcte.

6.3. SCÉNARIO DE TEST N°3 : VARIATION DE LA VOLUMÉTRIE DES DONNÉES

- **Identification** : TEST-FUNC-VOLUME
- **Description** : Vérifier que l'algorithme et l'importation traitent correctement des jeux de données de tailles différentes (cas minimal vs cas standard).
- **Procédure** :
 1. **Jeu de données réduit** : Charger un fichier contenant une série temporelle minimale (ex: seulement 2 ou 3 points) et une matrice P simplifiée.
 2. **Jeu de données normal** : Charger un fichier correspondant à une utilisation standard (ex: série de longueur N complète).
- **Résultat attendu** :

- L'application ne génère aucune erreur de segmentation, même avec un volume de données très faible.
- Le temps de traitement et l'affichage de la "Matrice P chargée" sont cohérents avec la taille du fichier importé.

6.4 . SCÉNARIO DE TEST N°4 : LOGIQUE D'ÉLAGAGE ET BACKTRACKING

- **Identification** : TEST-FUNC-CORE
- **Description** : Cœur de l'algorithme. Vérifier que l'arbre s'élague dès qu'un couple $((x[n-1], x[n]))$ est interdit par P.
- **Contraintes** : Pour chaque valeur $X_Q(n)$, tester les deux options (paire/impair).
- Si le compteur dans P est à 0, couper la branche.
- **Procédure** : Lancer la déquantification sur la série de test fournie , pour laquelle le résultat attendu est connu et sans ambiguïté .
- **Résultat attendu** : L'arbre doit converger vers la série originale. Les impasses doivent déclencher un retour en arrière (backtracking) pour tester l'autre branche.

6.5 . SCÉNARIO DE TEST N°5 : RENDU GRAPHIQUE DYNAMIQUE

- **Identification** : TEST-FUNC-GUI
- **Description** : Visualisation pédagogique du processus.
- **Procédure** : Exécuter le programme avec l'option graphique.
- **Résultat attendu** : Fenêtre affichant l'arbre binaire qui "pousse". Les branches élaguées doivent être visuellement identifiables (suppression du nœud).

7. CONFORMITÉ AUX SPÉCIFICATIONS D'INTERFACES

- **I-01** : Sortie des données au format **int** identique au format d'entrée pour comparaison simple.
- **I-02 : afficher** : Logs en temps réel indiquant le nombre de branches actives à chaque instant .

8. CONFORMITÉ DE LA DOCUMENTATION

- **DOC-01** : Code source commenté (Explication des structures de données de l'arbre).
- **DOC-02** : Manuel utilisateur succinct.

9. GLOSSAIRE

- **Quantification** : Réduction de la précision d'un signal.
- **Élagage** : Action de supprimer des branches invalides d'un arbre de recherche.
- **Série Temporelle** : Suite de valeurs numériques représentant un signal évoluant dans le temps.

.Validation finale

À l'issue des tests, une décision sera prise :

☐ GO – Validation du projet

☐ NO GO – Corrections nécessaires